

Começando com Git

Quando começamos a trabalhar com Git é comum nos depararmos com várias nomenclaturas novas. À primeira vista elas podem parecer confusas, mas com o uso e com as definições certas tudo se torna natural. Por isso, antes de executar qualquer comando, é fundamental entender como o Git funciona e quais são os elementos principais de um repositório.

Conceitos fundamentais

Repositório

Um repositório é onde o Git armazena tudo sobre o seu projeto: arquivos, histórico de mudanças, configurações, branches e versões anteriores.

Ele pode existir de duas formas:

- **Repositório local** – fica no seu computador; é onde você realmente trabalha, edita os arquivos e executa os comandos Git.
- **Repositório remoto** – fica na internet, em plataformas como GitHub ou GitLab; serve para backup, colaboração e compartilhamento do projeto.

Working Directory (Diretório de Trabalho)

É a pasta onde seus arquivos reais estão. É aqui que você abre os arquivos, altera códigos e cria novos documentos.

Staging Area

Imagine como uma “sala de preparação”. Quando você altera um arquivo, ele fica modificado, mas ainda não faz parte de nenhum registro oficial do Git.

Quando você usa `git add`, você coloca esse arquivo na staging area, dizendo ao Git:

“Isso aqui eu quero incluir no próximo commit.”

Commit

Um commit é como uma foto (snapshot) do estado dos arquivos naquele momento. Sempre que fazemos um commit, criamos uma versão nova e permanente do projeto, com:

- O que mudou
- Quem mudou
- Quando mudou
- Uma mensagem descrevendo a alteração

O commit é a menor unidade de mudança do Git.

Configurando o Git

Instalando Github

Primeiro vamos intalar o Git no computador. No linux atualize o apt e installe o Github.

```
sudo apt update
sudo apt install git
```

Criando conta no Github

O host de repositórios remotos. [Github](#)

Configurando o usuário Git

Depois de criar a conta no Github, você precisa no seu computador criar o seu usuario git

```
git config --global user.name "Seu Nome"
git config --global user.email "seuemail@example.com"
```

Outra config importante é dizer pro git como queremos chamar a branch principal nossa. É comum chamar de **master** em projetos mais antigos ou **main** mais recente. Vamos configurar para o padrão ser **main**:

```
git config --global init.defaultBranch main
```

Meu primeiro repositório

Com esses conceitos claros podemos começar. Para criar um repositório local do zero usamos o comando abaixo, que transforma **qualquer pasta** em um repositório Git:

```
git init
```

So use esse comando quando deseja transformar a pasta que você está em um repositório Git. Eu pessoalmente gosto de guardar meus repositórios em uma pasta dentro da pasta Documents (**Documents/repo/**).

Entra em Documents e cria a pasta **repo**:

```
cd Documents
mkdir repo
```

Entre na pasta `repo` e cria uma pasta `meu_primeiro_repo`

```
cd repo
mkdir meu_primeiro_repo
```

Agora vamos tornar essa pasta um repositório Git:

```
git init
```

Agora, vamos adicionar um novo arquivo nesse repositório. E colocar o código de hello world de python.

```
nano meu_codigo.py
```

Escreva esse código dentro do arquivo e feche com Ctrl+X e depois Y pra salvar com o nome escolhido.

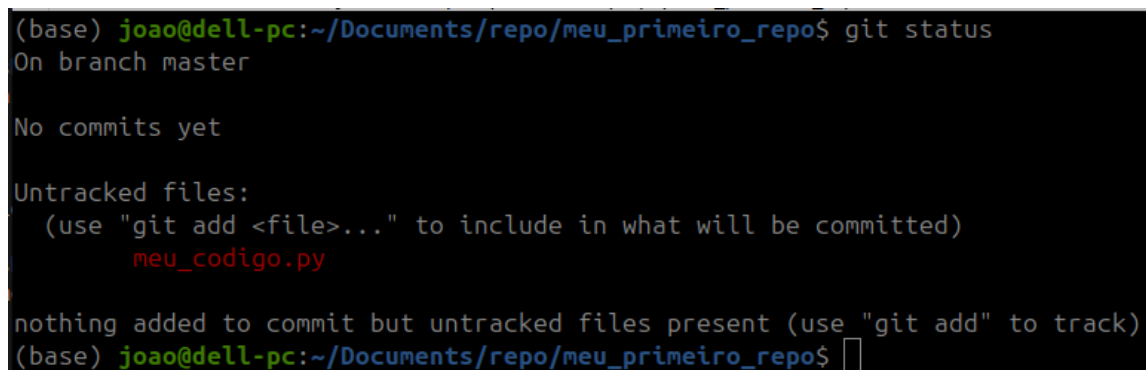
```
print("Hello, World")
```

Podemos verificar listando os arquivos da pasta com `ls`.

O segundo comando Git que vamos aprender é o para verificar o status do repositório

```
git status
```

Você deverá ver algo como a imagem abaixo:

A terminal window with a dark background. The prompt is (base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo\$. The command git status has been executed. The output shows 'On branch master', 'No commits yet', and 'Untracked files:' followed by '(use "git add <file>..." to include in what will be committed)' and 'meu_codigo.py' in red. Below this, it says 'nothing added to commit but untracked files present (use "git add" to track)'. The prompt is now (base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo\$ with a cursor.

```
(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    meu_codigo.py

nothing added to commit but untracked files present (use "git add" to track)
(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo$
```

Nessa print do terminal, podemos ver que o arquivo que criamos fica em vermelho na seção de Untracked files, ou seja, é um arquivo que foi alterado/adicionado que não existia em uma outra versão do repositório.

Meu primeiro Commit

Antes de fazer nosso primeiro commit. Vamos criar mais um arquivo, mas agora um arquivo README.

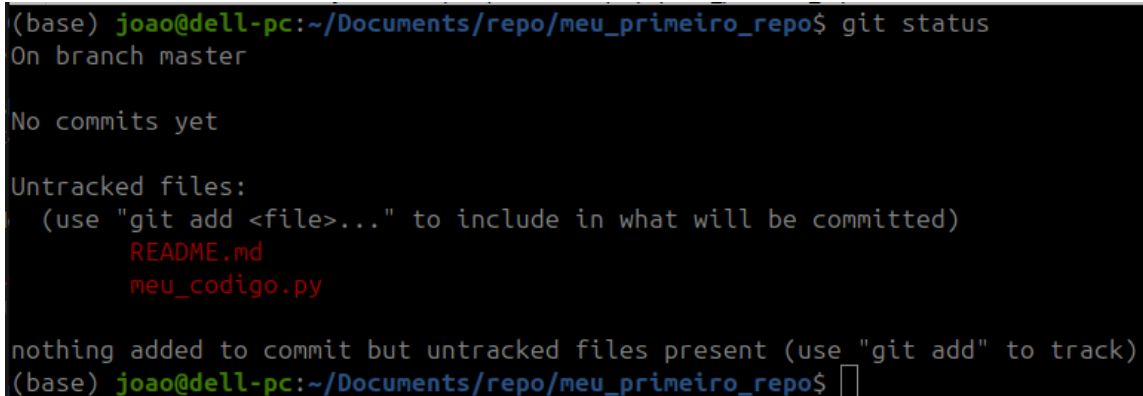
```
nano README.md
```

Os arquivos Markdown (.md) são muito utilizados pra códigos e documentação. Ele é um arquivo de texto formatado, tipo uma versão mais simples de html (inclusive ele aceita html). Nele você pode adicionar títulos, tabelas, imagens de uma forma mais simples e direta. Vamos colocar uma breve mensagem no nosso README:

```
# Meu primeiro repositório

Esse é meu primeiro README!
Meu nome é : <NOME>
Meu curso é: <CURSO>
```

Feche e salve o arquivo. Novamente vamos dar um `git status`. Podemos observar que o arquivo README foi também pra área do Untracked files:

A terminal window with a black background and green text. The prompt is '(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo\$'. The command 'git status' has been executed. The output shows 'On branch master', 'No commits yet', and 'Untracked files: (use "git add <file>..." to include in what will be committed)'. Under 'Untracked files', 'README.md' and 'meu_codigo.py' are listed in red. At the bottom, it says 'nothing added to commit but untracked files present (use "git add" to track)'. The prompt is now '(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo\$' with a cursor.

```
(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
       README.md
       meu_codigo.py

nothing added to commit but untracked files present (use "git add" to track)
(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo$
```

Vamos agora fazer nosso primeiro commit, nele não vamos mandar o código, somente o README.

Para fazer o commit precisamos falar pro Git quais arquivos queremos adicionar no nosso commit. Pra isso existe o comando `git add <arquivos>`. Ele move os arquivos de untracked para a *staging area*.

No `git add` você tem que fornecer o caminho dos arquivos que deseja adicionar. No nosso caso:

```
git add README.md
```

Usando o `git status` podemos observar que o README.md foi movido para a *staging area* por ele ter mudado de cor. Inclusive o Git também indica que ele é um *new file*.

```
(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    meu_codigo.py

(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo$
```

No `git add` você também pode passar um caminho pra uma pasta para adicionar a pasta inteira ou também rodar `git add .`, que indica ao Git pra adicionar todas as alterações.

Agora vamos enviar a alteração nossa pro repositório local usando o comando `git commit`. Para commits pequenos é comum utilizar:

```
git commit -m "Minha mensagem de commit"
```

O `-m` indica que você tá enviando a mensagem de commit a seguir.

Contudo, é preferível utilizar mensagem de commits mais detalhadas sempre que possível, indicando o que foi feito, porque foi feito. Nisso basta usar o commando sem o parametro `-m`:

```
git commit
```

Ele vai abrir um editor identico ao nano onde você pode adicionar a sua mensagem de commit. Na primeira linha é a descrição mais genérica de commit e depois nas linhas seguintes uma descrição mais detalhada:

```

/home/joao/Documents/repo/meu_primeiro_repo/.git/COMMIT_EDITMSG *
Adding the README file

This is my first commit. I am adding the README file to my repo.
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
#
# Initial commit
#
# Changes to be committed:
#   new file:   README.md
#
# Untracked files:
#   meu_codigo.py
#
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line

```

Feche com Ctrl+X, salve com Y e deixe o nome do arquivo como está.

Após isso se digitar `git status`:

```

(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo$ git commit
[master (root-commit) 086b100] Adding the README file
1 file changed, 3 insertions(+)
create mode 100644 README.md
(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
      meu_codigo.py

nothing added to commit but untracked files present (use "git add" to track)
(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo$

```

Pronto agora você já fez o seu primeiro commit. Como prática agora faça o mesmo passo a passo só que adicionano o arquivo `meu_codigo.py`

Linkando um repositório remoto com um repositório local

Configurando uma chave SSH no git hub

Para conseguir se conectar com o repositório remoto precisamos de uma forma de autenticação, as mais comuns são por tokens e por chaves SSH. A mais prática e a mais utilizada são as chaves SSH, por isso vamos utilizar elas. O Github já tem um tutorial bem estabelecido deles de como adicionar, você pode seguir ele:

- [Adicionando uma chave SSH no Github](#)

Conectando o repositório local com o remoto

Com o repositório local já criado e com commits feitos, o próximo passo é criar um repositório remoto no GitHub e “linkar” os dois.

Criando o repositório no GitHub

1. Acesse o GitHub e faça login.
2. Clique em New repository.
3. Escolha um nome (ex: meu_primeiro_repo).
4. Não marque as opções de criar README, .gitignore ou licença (pois já temos um repositório local).
5. Clique em Create repository.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * **Repository name ***

JPBG-USP /

Great repository names are short and memorable. How about [super-duper-fishstick?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility * Choose who can see and commit to this repository Public

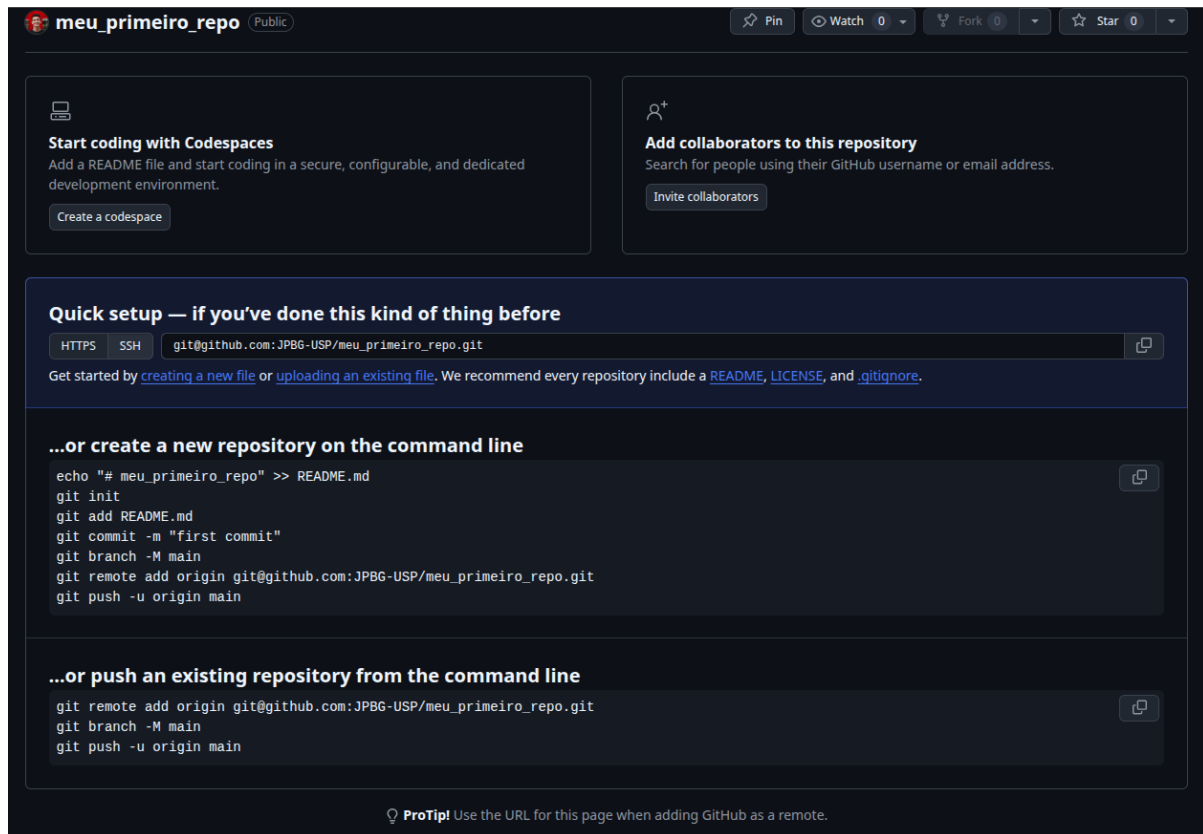
Add README READMEs can be used as longer descriptions. [About READMEs](#) Off

Add .gitignore .gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

Add license Licenses explain how others can use your code. [About licenses](#) No license

Create repository

Após isso, o GitHub irá mostrar a página do repositório vazio, com algumas instruções. Vamos usar a opção de repositório existente.



Você pode executar os comandos em "...or push an existing repository from the command line" ou seguir os passos abaixo:

No terminal, dentro da pasta do seu repositório local (**meu_primeiro_repo**), execute:

```
git remote add origin git@github.com:SEU_USUARIO/meu_primeiro_repo.git
```

1. **git remote add** - adiciona um repositório remoto
2. **origin** - nome padrão do repositório remoto
3. O link **git@github.com:...** é o endereço SSH do repositório no GitHub

Você pode verificar se o remoto foi configurado corretamente com:

```
git remote -v
```

Agora vamos enviar os commits do repositório local para o GitHub usando o comando **git push**.

Se for o primeiro push, utilize:

```
git push -u origin main
```

1. **push** - envia os commits para o remoto

2. `-u` - define a branch remota como padrão
3. `origin` - nome do repositório remoto
4. `main` - nome da branch

Após esse comando, seus arquivos e commits aparecerão no GitHub e você realizou o seu primeiro commit.

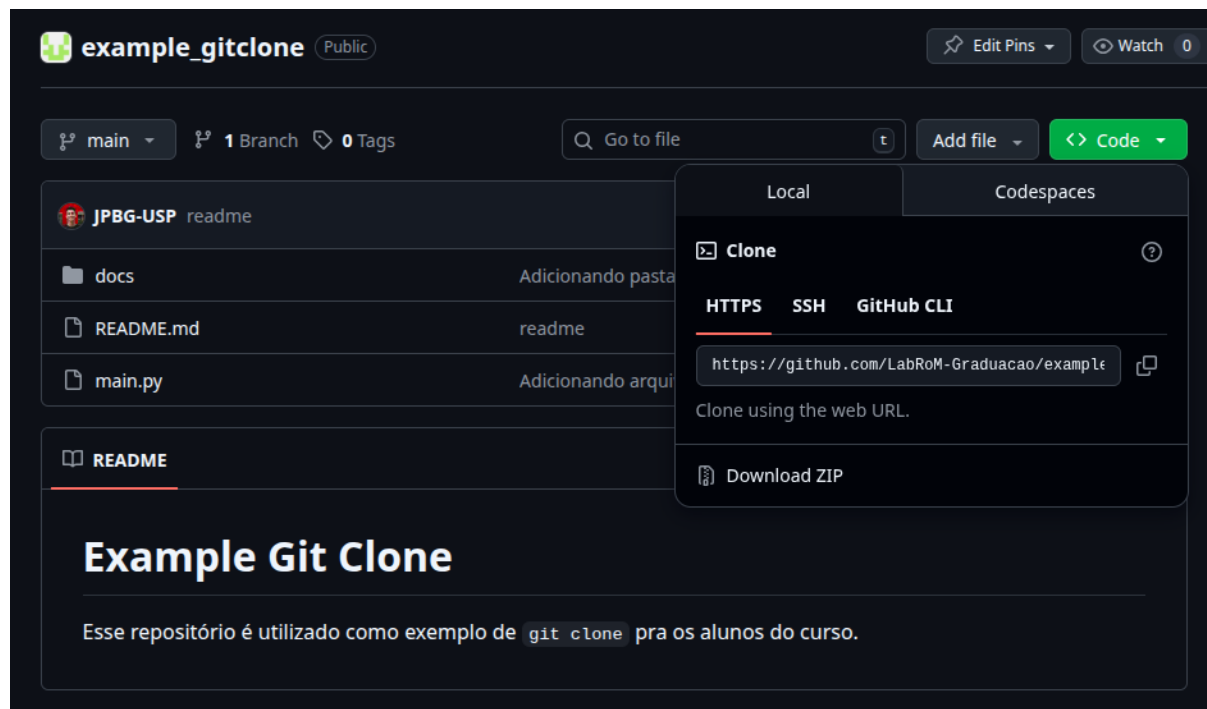
Clonando um repositório

Vimos como criar o nosso repositório do zero, mas em alguns casos vamos ter que usar um repositório já criado por alguém. Nesses casos utilizamos:

```
git clone <URL>
```

No github há duas principais formas de obter o URL do repositório para clonar ele, o HTTPS e o SSH. Se você ir fazer alterações no repositório o recomendado é utilizar o SSH, já que se utilizar o HTTPS todo push que fizer terá que fornecer o token de acesso, o que não é preciso se utilizar o SSH. Agora se for somente utilizar o repositório pronto do jeito que está, utilize o HTTPS, já que com ele se o repositório for publico, até pessoas sem acesso de administrador podem baixar.

No Github, clicando em code ira aparecer as opções para você selecionar. Como exemplo estamos utilizando uma repositório feito para essa aula.



Podem clonar no computador de vocês usando o seguinte comando. (Usamos HTTPS)

```
git clone https://github.com/LabRoM-Graduacao/example_gitclone.git
```

Atualizando o repositório local

Como ultimo comando dessa aula, vamos ver como baixar as atualizações do repositório remoto para o repositório local.

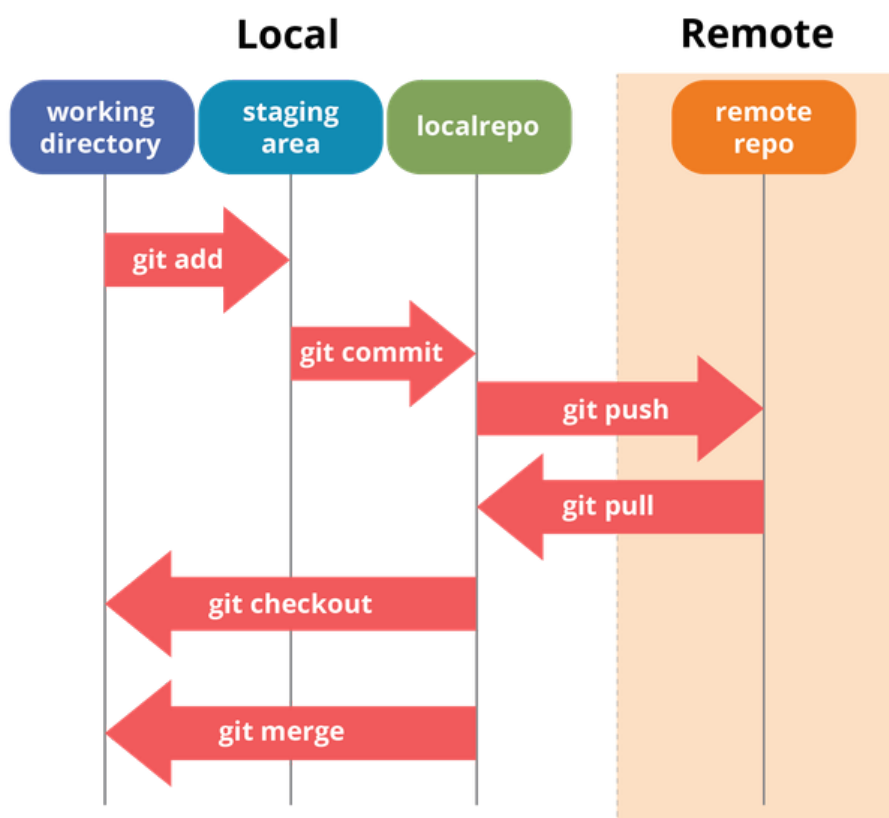
```
git pull
```

Esse comando é mais utilizado quando trabalhando em equipe, e o repositório remoto não acompanha o seu repositório local. Coisas que vamos ver nas próximas aulas como fazemos.

Revisão de Comandos

Nessa aula aprendemos os comandos mais utilizados no Git, vimos como criar um repositório local e conectar em um repositório remoto. Também vimos como fazer um commit e enviar as alterações.

Abaixo trago uma imagem que exemplifica o que cada comando faz. Inclusive nela também tem dois comandos que veremos na próxima aula, o `git checkout` e o `git merge`.



Comandos básicos do Git